SafeRL-Drive Internal Experiment Record

Dheeraj Dhillon

22 July 2026

Abstract

This surrogate is the technical record behind the compact SafeRL-Drive report. It retains failed runs, causal debugging, exact protocol, commands, persistence rules, and the predeclared traffic-pilot decision. The public report presents the result; this document preserves why each result can or cannot be trusted. Traffic adaptation has been implemented but not yet trained at the time of this revision.

1 Project boundary

The final question is whether a geometry-competent SAC policy can adapt to interactive procedural traffic while reducing collision and retaining traffic-free generalization. This is single-agent RL: SAC controls one ego car and MetaDrive controls every background vehicle. Every final configuration fixes `num_agents=1` and `is_multi_agent=false`. MultiAgentMetaDrive, self-play, learned background vehicles, image-policy training, new algorithms, architecture searches, and broad reward sweeps are excluded. MetaDrive provides the procedural simulator and native traffic manager [2]; SAC is the sole final learned controller family [1].

The experiment stops after three new long runs: seed-0 reference traffic adaptation, seed-0 safety-penalty adaptation, and seed-1 confirmation of the declared winner. IDM, ExpertPolicy, frozen-source evaluations, rendering checks, and videos do not count as training runs.

2 Historical audit: what failed first

The first ambitious Phase-1 task combined a five-block road, density-0.1 traffic, weak terminal penalties, long horizons, and unstable checkpoint selection. PPO made progress but usually left the road. SAC learned a profitable slow or stalled behavior and later exhibited critic and entropy instability. In one audited run, SAC's critic loss rose from below one to millions, the learned entropy coefficient grew above 13, and the final policy mostly braked or saturated its actions.

The audit found four measurement defects:

1. MetaDrive's `velocity` field is already km/h; multiplying by 3.6 inflated legacy speed reports.
2. Validation and claimed test scenarios overlapped.
3. Mean reward selected checkpoints even when success and route completion declined.
4. Environment creation and evaluation could share MetaDrive's process-global engine.

Corrections separated train, validation, and test seeds; selected checkpoints by success, route completion, and failure rates; retained raw action and terminal information; and placed validation in its own simulator subprocess. MetaDrive's default vector observation was kept unnormalized because it is already bounded.

3 Phase 1: infrastructure control

The corrected Phase-1 objective was deliberately easy: demonstrate that both a discrete PPO control and continuous SAC control could complete a straight traffic-free road using frozen held-out policies. Training could stop only after a success-first validation gate. IDM, PPO, and SAC each achieved 100% success on 20 test scenarios, with zero collision and off-road outcomes. The 20-of-20 Wilson lower bound is only 83.9%, so this was a pipeline gate rather than robustness evidence.

Action telemetry changed the interpretation. PPO's deterministic solution was essentially zero steering plus full throttle: a valid but trivial open-loop solution. SAC applied small steering corrections and varied throttle/braking. Consequently SAC became the continuous-control basis for procedural geometry. PPO remains a historical control and is not retrained with traffic.

4 Phase 2: procedural geometry

Direct SAC and curriculum SAC shared a $[256, 256]$ MLP, full continuous actions, reference MetaDrive reward, 500,000 maximum steps, no traffic, 25 validation scenarios, and 100 held-out test scenarios. Direct SAC trained on three-block maps. Curriculum learning was used as a staged optimization intervention rather than a new algorithm family [3]. Curriculum SAC trained on `C`, then `SC`, then the identical three-block target while preserving model and replay state.

Seed-level success was 24% versus 86% for seed 0 and

Table 1: Final verified Phase-2 means across seeds 0 and 1.

Condition	Succ.	Coll.	Off-road	Route
Direct	12.0%	15.5%	83.0%	47.0%
Curriculum	87.0%	0.0%	9.5%	95.1%

0% versus 88% for seed 1. Curriculum seed 0 stopped after 150,000 total steps when its validation gate passed. Curriculum seed 1 peaked before the final update: validation reached 92% success at 375,000 steps and declined later, while the frozen best checkpoint still achieved 88% on test. Success-first checkpoint preservation was therefore essential.

4.1 Retained failed launches

Direct seed 2 completed 500,000 steps but achieved 0% test success, 100% off-road, and 19.2% route completion. Curriculum seed 2 failed the mandatory curve stage after 100,000 steps; its best validation checkpoint had 0% success and 41.8% route completion. These runs were not silently rerun and are not included in the two complete pairs. They demonstrate initialization sensitivity and limit the geometry claim.

4.2 Native ceilings

On the exact Phase-2 test, IDM achieved 95% success and ExpertPolicy 100%. Native controllers are feasibility ceilings, not learned-policy equivalents. Curriculum SAC approached IDM without exceeding the simulator's native control.

5 Historical artifact ledger

The audit used the repository's saved summaries, per-episode CSVs, latest pointers, and notebook outputs rather than reconstructing results from memory. The following ledger is the evidence retained before the traffic extension:

Phase	Condition	Seeds	Episodes/seed
1	IDM/PPO/SAC control	one each	20
2	Direct SAC complete	0, 1, 2	100
2	Curriculum SAC complete	0, 1	100
2	Curriculum failed gate	2	validation only
2	Native IDM/Expert	shared test	100
2	Zero-shot traffic learned	0, 1	100

"Complete" means that the run has an immutable resolved configuration, selected model, and compatible held-out episode record. It does not mean that the policy performed well. Direct seed 2 is deliberately retained despite total failure. Curriculum seed 2 is retained even though it never advanced beyond the curve gate. These artifacts prevent a clean two-seed table from erasing the observed optimization instability.

Table 2: Traffic curriculum.

Stage	Density	Max new steps	Gate
A	0.02	100,000	required
B	0.05	200,000	descriptive

The final comparison does not pool the Phase-1 easy-road episodes, Phase-2 procedural episodes, or zero-shot density-0.05 episodes. Their maps, traffic definitions, selection roles, and sometimes action interfaces differ. They are historical evidence only. The new evaluator may reuse a prior episode file solely when its strict fingerprint encodes the exact same environment, reward, action, split, controller, and episode count.

6 Why traffic is the final extension

Frozen Phase-2 policies were tested zero-shot at density 0.05:

Condition	Succ.	Coll.	Off-road	Route
Direct	3.5%	37.0%	67.5%	37.3%
Curriculum	69.0%	21.0%	6.0%	87.1%

The curriculum result shows useful initialization: geometry and lidar processing transfer partially. It is not trained traffic competence. Seed variation was material: curriculum seeds 0 and 1 achieved 59% and 79% success, with 34% and 8% collision. Traffic exposure is therefore a stronger final story than another algorithm or larger network.

7 Final experiment protocol

7.1 Solvability and source freeze

IDMPolicy and ExpertPolicy first run 50 scenarios at densities 0.05 and 0.10, start seed 50000, map 3, horizon 1000, respawn traffic, and fixed traffic realization. A short ExpertPolicy chase video is a rendering gate.

The completed geometry-curriculum checkpoints from seeds 0 and 1 are then evaluated without retraining at densities 0, 0.05, and 0.10. Each condition uses 100 test scenarios from seed 60000. Results are reused only when their full fingerprint matches.

7.2 Traffic stages

Both seed-0 pilots branch from the exact same curriculum seed-0 best checkpoint. Model policy state transfers; the source replay buffer and optimizer state do not. A new SAC model is constructed with the traffic configuration, actor/critic/target weights are loaded from the source policy state, and a fresh replay buffer starts at size zero.

Both stages use respawn traffic and randomized traffic during training. Stage A must reach success ≥ 0.70, collision ≤ 0.25, off-road ≤ 0.15, and route completion ≥ 0.80 on 25 fixed validation episodes. Failure is terminal for automatic progression. No replacement pilot is launched.

7.3 Controlled reward ablation

SAC-Traffic keeps vehicle/object collision penalties at 5. **SAC-Traffic-Safe** changes those two values to 10. Success reward remains 10; off-road and sidewalk penalties remain 5; driving reward is 1; speed reward is 0.1; lateral reward and custom shaping are disabled. No TTC term or safety shield is added.

7.4 SAC configuration

Setting	Value	Setting	Value
Policy	MLP	Layers	256,256
Learning rate	10^{-4}	Batch	512
Buffer	300,000	Starts	5,000
γ	0.99	τ	0.005
Train frequency	1	Target interval	1
Entropy	0.05	Device	auto
Timeout handling	true	Compact memory	false

The notebook derives $N = \min(4, \max(1, \text{CPUCount} - 1))$, uses one MetaDrive engine per subprocess, and sets gradient steps to N. This keeps the approximate update-to-new-transition ratio near one. The goal is simulator throughput and stable learning, not artificial GPU saturation.

8 Resolved traffic configuration ledger

The authoritative file is `configs/sac_traffic_curriculum.yaml`. One YAML serves both pilots; `--variant` changes only the declared reward entries. The common training environment is:

Key	Value	Key	Value
map	3	scenarios	200
start seed	40000	sequential seed	true
spawn lane random	false	horizon	1000
truncate as terminal	false	action	continuous
traffic mode	respawn	random traffic	true
accident probability	0	image observation	false
crash vehicle done	true	crash object done	true
off-road done	true	agents	1
multi-agent	false	render	false

Traffic density is the only stage-wise environment override: 0.02 for at most 100,000 new transitions and 0.05 for at most 200,000. Training traffic is randomized; validation and test traffic use fixed realizations. Full steering remains available. The default bounded MetaDrive state/lidar observation is retained, and normalization of observation and reward is disabled.

Split	Start seed	Scenarios	Densities
Training	40000	200 cycling	stage value
Gate validation	50000	25	stage value
Pilot selection	50000	50	0, .05, .10
Final test	60000	100	0, .05, .10

All evaluation inference is deterministic. Pilot selection and final test share density values but not episode counts or scientific roles. The selection matrix chooses the reward variant; the held-out test matrix estimates its final behavior.

The common reward ledger is success 10, off-road 5, sidewalk collision 5, driving 1, speed 0.1, lateral reward disabled, and an empty custom shaping dictionary. Reference vehicle and object collision penalties are 5; safety penalties are 10. No other reward key changes. Both conditions use the same SAC implementation and $[256, 256]$ network from Stable-Baselines3 [4].

Operational frequencies are evaluation every 25,000 new transitions, checkpoint every 50,000, diagnostics every 10,000, and resource sampling every 60 seconds. Replay saving and the SB3 progress bar are enabled. Training verbosity is zero. The config carries `n_envs=1` and `gradient_steps=1` as portable defaults; the canonical notebook resolves both together at runtime and freezes the resolved values in the run.

8.1 Pilot selection

Both seed-0 pilots run a 50-scenario validation matrix at densities 0, 0.05, and 0.10. A pilot qualifies at 0.05 with success ≥ 0.80, collision and off-road ≤ 0.10, route completion ≥ 0.90, and traffic-free success no more than ten points below the source.

If exactly one qualifies, it wins. If both qualify, lower collision wins; within two collision points, higher success wins; a remaining tie uses route completion. If neither qualifies, higher success wins; within five success points, lower collision wins. The decision and compared values are written to `runs/traffic_extension_selection.json`. Only that variant is run from the seed-1 geometry checkpoint. There is no automatic seed 2.

9 Evaluation record schema

Final evaluation uses the same 100 scenario IDs for source SAC, adapted SAC, IDMPolicy, and ExpertPolicy at each density. Each episode stores training seed, density, condition, checkpoint, fingerprint, return, length, cost, speed, steering magnitude, action change, route completion, aggregate collision, vehicle and object collision, off-road, timeout, and success.

Raw terminal flags are non-exclusive. A plotting field imposes priority: success, collision, off-road, timeout, other. This prevents stacked outcome figures from exceeding 100% while preserving simulator detail. Summaries

report a Wilson 95% success interval. Final effects are density-0.05 success gain and collision change, density-0 success and route retention, and adapted degradation from 0.05 to 0.10. Two-seed means and sample standard deviations are reported, with no real-world safety claim.

9.1 Metric definitions and interpretation

Success, crash, vehicle crash, object crash, off-road, and maximum-step indicators are copied from the terminal MetaDrive information. Aggregate collision is the saved crash flag; `crash_vehicle` remains separate because moving-traffic contact is the primary hazard. Collision-free is the complement of aggregate crash, not a claim of safe driving. A timeout includes an episode ending at the configured horizon without another higher-priority terminal outcome.

Route completion is the terminal fraction of the planned route. Mean return is the environment reward accumulated over the episode and therefore must be interpreted under the recorded reward variant. Mean cost is MetaDrive's accumulated cost signal. Mean speed uses the simulator's velocity value directly in km/h; the historical erroneous factor of 3.6 is not applied. Steering magnitude averages the absolute first action coordinate. Action change averages consecutive action-vector differences and is a rough smoothness diagnostic, not a comfort model.

The Wilson interval is computed over episode success indicators for each condition. Across-seed standard deviation is computed only after producing one condition summary per training seed. These two uncertainty summaries answer different questions: episode sampling at a fixed policy versus variation between two training lineages. Neither supports a population-level safety statement.

The terminal-outcome priority prevents double counting. A successful terminal transition is labeled success even if a lower-priority raw flag is also present; otherwise collision precedes off-road, which precedes timeout, then other. The raw flags remain available so this plotting convention can be audited or changed without rerunning the simulator.

10 Warm start and resume semantics

The traffic runner resolves the seed-specific `latest_sac_phase2_curriculum_seedN.txt` pointer, normally selects `best_model.zip`, and records its SHA-256 digest, source evaluation fingerprint, source run path, and source summary in `source_lineage.json`. It validates observation class, shape, dtype, bounds, and the same action properties before loading weights. A mismatch fails before learning.

Fresh source replay is the default. Importing source replay requires the explicit `--load-source-replay-buffer` switch and is not used by the canonical notebook. After traffic learning begins, its own resumable model and traffic replay buffer are saved at each stage boundary. Resume requires exact equality with the immutable `resolved_config.yaml`, including reward variant, environment count, and gradient steps.

State records total new transitions rather than inheriting the source model's old timestep count. Stage records also include density, traffic mode, scenario range, elapsed wall time, and approximate environment FPS.

10.1 Run state and artifact contract

A new traffic launch first resolves the source pointer and model, verifies spaces, creates a new timestamped directory, writes its immutable config and lineage, and only then initializes learning. During a stage, periodic checkpoints are diagnostic; they are not silently promoted over the success-first validation choice. At a stage boundary the runner saves a resumable model and replay buffer before updating curriculum state.

The state machine has four externally meaningful outcomes:

State	Notebook behavior
running/interrupted	restore model and traffic replay, then resume
stage complete	continue at the next declared stage
failed gate	preserve and stop; never bypass automatically
complete	reuse; never overwrite or duplicate

Resume fails when the live override changes the source, variant, number of environments, vectorization mode, gradient steps, stage budgets, or another immutable field. This strictness is intentional: accepting a seemingly harmless runtime change could alter the number of optimizer updates per transition and invalidate the controlled comparison.

Each completed run must contain `resolved_config.yaml`, `run_metadata.json`, `curriculum_state.json`, `source_lineage.json`, a resumable model and replay buffer, and at least one selected or final model. Evaluation adds episode CSVs, summaries, and fingerprints under `eval/`; detailed training and resource records stay under `logs/`. The latest pointer is outside the run directory so consumers can resolve one canonical lineage without scanning timestamps.

11 Pinned chase-camera implementation

The repository pins MetaDrive commit `85e5dadc6c7436d324348f6e3d8f8e680c06b4db`. At that commit, retaining a `main_camera` sensor with `use_render=false` activates offscreen rendering. After reset, MetaDrive tracks the ego with the

chase task. The recorder obtains CPU frames with `env.main_camera.perceive(to_float=False)`.

MetaDrive otherwise filters the main camera from its sensor list when both normal rendering and image observation are disabled. The pinned implementation therefore uses MetaDrive's image-service switch only to retain the offscreen camera and simultaneously sets `agent_observation=LidarStateObservation`. SAC still receives the trained vector, never the RGB frame. Camera settings are 1280×720, distance 8.5, height 2.8, FOV 65, smoothing and follow-lane enabled, with interface clutter hidden. No top-down fallback occurs. Errors report offscreen mode, sensor names, expected/received shape, MetaDrive version, and pinned commit.

The macOS smoke test exposed a second pinned-version defect: `MainCamera` calls the window-only `requestProperties` method when it tries to hide the cursor, but an offscreen `GraphicsBuffer` has no such method. The recording environment leaves the unused mouse-display flag enabled, which skips that call while keeping every visible interface panel disabled. A five-step ExpertPolicy smoke then produced real frames and confirmed a 259-element vector policy observation.

The Colab L4 smoke exposed a separate display-backend failure: the vector environment completed, but the display-less Linux chase process exited before producing a frame. Forcing Panda3D's EGL-backed `p3headlessgl` pipe selected the intended backend, but the native process still terminated without a Python traceback on the Colab L4 runtime. MetaDrive's own Linux rendering tests use an X virtual framebuffer. The recorder now follows that route: on Linux without a `DISPLAY`, it automatically relaunches itself under `xvfb-run` and forces Mesa software GL for predictable video generation. Training remains GPU-enabled; only the small rendering process uses this compatibility path. Linux systems with a display and macOS retain the platform default. The notebook installs `xvfb`, `xauth`, and the Mesa GL runtime when missing and checks every shell command's exit code immediately, so a compile, test, vector-smoke, or chase-smoke failure stops at its real source instead of surfacing later as a missing-file assertion. Chase diagnostics include the active graphics pipe and window type.

A second Colab attempt proved that backend selection alone was insufficient. The process entered Xvfb display `:99`, printed Panda3D errors because the minimal container had no `/dev/input`, and terminated with native exit code 139 before its first frame. This rules out a Python exception and rules out the earlier EGL-only diagnosis. That failing image had also resolved a newer native stack than the pinned MetaDrive snapshot: NumPy 2.0.2, OpenCV 5.0.0.93, and Panda3D 1.10.16. The repository now pins NumPy 1.26.4, OpenCV 4.11.0.86, and Panda3D 1.10.15. Notebook Section 7 repairs this stack even after a mid-notebook resume, verifies imports in a fresh subprocess, creates an empty `/dev/input`, selects Mesa `llvmpipe`, and enables Python fault handling in the renderer child. This is a compatibility correction awaiting the next Colab frame-producing smoke, not yet evidence that frame capture passed on Colab.

Scenario selection is deterministic from episode CSVs: the lowest matching seed is used for first, first-success, or first-failure. Every MP4 sidecar records source, seed, density, mode, outcome, return, completion, collision, frame count, FPS, camera settings, vector observation shape, and fingerprint. The required set is a matched before/after pair, a representative adapted success, an adapted failure when one exists, and an expert demonstration.

12 Progress, logging, and profiling

SB3 owns the only training progress bar. The runner exposes `--progress` and `--no-progress`; no outer tqdm wraps learning. Each stage prints its name, density, maximum budget, and concise validation result. Evaluation uses one tqdm loop when enabled. SB3 verbose output remains zero.

Console logging is concise INFO; files retain DEBUG context and stack traces. Startup metadata includes command, resolved config, working directory, Git commit, Python and OS, package versions, CPU and RAM, CUDA/GPU name and memory, selected device, environment count, observation/action shapes, lidar beams, and policy/actor/critic parameter counts. `logs/resource_usage.csv` samples CPU, RAM, GPU utilization and memory when `nvidia-smi` is present, plus environment steps per second, approximately every 60 seconds.

13 Implementation verification record

No long traffic adaptation was run during implementation. Static compilation succeeded for `saferl_drive`, `scripts`, and `tests`. The exact `pytest -q` invocation passed all 54 tests. The suite covers traffic configuration invariants, stage resolution, warm-start lineage, fresh-replay behavior, immutable resume checks, density matrices, exclusive outcomes, latest pointers, fingerprints, selection, Drive critical files, video sidecars, and notebook structure.

A local five-step vector-observation traffic smoke used one learned ego slot and native background traffic and returned the expected 259-element observation. A separate five-step ExpertPolicy chase smoke produced an MP4 from actual main-camera frames after the pinned macOS workaround; its sidecar recorded density 0.05 and the same vector observation shape. An independent 16-step SB3 toy run displayed the live `progress_bar=true` output, verifying both `tqdm` and `rich`.

These checks establish importability, simulator construction, traffic spawning, vector policy input, frame production, encoding, and orchestration. They do not establish

that the three long SAC runs learn. CUDA execution, L4 throughput, multi-process MetaDrive training over the full budgets, Stage-A qualification, pilot selection, seed-1 confirmation, and final statistics remain Colab responsibilities.

14 Colab and Drive record

The failing Phase-2 render runtime was Google Colab with an NVIDIA L4 on 22 July 2026: Python 3.12.13, MetaDrive 0.4.3, Stable-Baselines3 2.9.0, Gymnasium 1.3.0, PyTorch 2.11.0/CUDA 12.8, NumPy 2.0.2, OpenCV 5.0.0.93, and Panda3D 1.10.16. The canonical renderer now uses the compatibility pins above. New runs record their actual environment rather than assuming these versions.

The live Git clone is `/content/safedrive`; persistent storage is `/content/drive/MyDrive/SafeDrive`. Code never runs from mounted Drive. A run sync copies one immutable run to Drive and then verifies config, metadata, curriculum state, lineage, resumable model, replay buffer, and at least one selected/final model. Small metadata and pointers use temporary-file replacement. TensorBoard events and intermediate checkpoints are excluded from project-level sync. The Mac restore remains analysis-only unless training artifacts are explicitly requested.

15 Canonical notebook

Only `notebooks/phase2_colab_driver.ipynb` remains. Its sections are:

0. direction and experiment policy;
1. constants and paths;
2. runtime inspection;
3. Drive mount;
4. clone or fast-forward pull;
5. install and version verification;
6. Drive restore;
7. compile, tests, traffic and chase smoke gates;
8. historical Phase-1/2 summaries;
9. native traffic solvability;
10. frozen-source matrix;
11. reference seed-0 pilot;
12. safety seed-0 pilot;
13. selection;
14. selected seed-1 confirmation;
15. final matrix;
16. third-person videos;
17. plots and tables;
18. report builds;
19. final Drive sync.

Training cells inspect latest pointers. Completed runs are reused, paused runs resume with their saved environment count, failed gates remain terminal, and unknown states fail closed. Main entry points appear as visible `!python -m` commands.

15.1 Restart playbook

After a Colab disconnect, rerun Sections 1–7 only. They recreate the live clone, install the pinned package, mount Drive, restore runs and pointers, and repeat cheap gates. Then inspect the next long section. Its helper reads the latest pointer and metadata: a completed match is skipped, an interrupted match emits the exact resume command, and a failed gate remains visible. The helper does not decide that an unknown directory is safe to continue.

The selection step is restart-safe independently of Python variables because its full inputs and decision are serialized to `traffic_extension_selection.json`. Seed 1 reads the selected variant from that file. Likewise, final comparison resolves runs through persisted pointers rather than depending on objects left in notebook memory.

The live clone remains disposable. Drive is the authority for completed training artifacts, while Git is the authority for code. Project restore does not make mounted Drive the working directory. The final sync separately copies compact comparison files, videos and sidecars, TeX sources, bibliography, generated macro fragments, and PDFs.

16 Exact commands

```
python -m scripts.train_curriculum \
 --config configs/sac_traffic_curriculum.yaml \
 --variant reference --seed 0 \
 --n-envs 4 --vec-env subproc --progress

python -m scripts.train_curriculum \
 --config configs/sac_traffic_curriculum.yaml \
 --variant safety --seed 0 \
 --n-envs 4 --vec-env subproc --progress

python -m scripts.evaluate --run-dir <pilot> \
 --model best --split validation --episodes 50 \
 --densities 0.0 0.05 0.10 --prefix traffic_pilot \
 validation.num_scenarios=50

python -m scripts.compare_runs \
 --traffic-extension --select-pilots

python -m scripts.train_curriculum \
 --config configs/sac_traffic_curriculum.yaml \
 --variant <selected> --seed 1 \
 --n-envs 4 --vec-env subproc --progress

python -m scripts.evaluate --run-dir <run> \
 --model best --split test --episodes 100 \
```

```
 --densities 0.0 0.05 0.10 \
 --prefix traffic_final

python -m scripts.compare_runs --traffic-extension

python -m scripts.sync_drive_runs \
 --drive-project /content/drive/MyDrive/SafeDrive \
 --to-drive --run-dir <run>
```

If Colab resolves fewer than four environments, substitute the notebook's saved value. Resume must reuse the immutable value stored with the run.

17 Expected output inventory

Each adapted run contains resolved config, run metadata, source lineage, curriculum state, stage/final/best models, resumable traffic replay, validation histories, episode CSVs, summaries, monitor logs, resource samples, plots, and video sidecars. Project outputs are:

```
runs/traffic_extension_seed_results.csv
runs/traffic_extension_comparison.csv
runs/traffic_extension_comparison.json
runs/traffic_extension_selection.json
runs/traffic_extension_outcomes.png
runs/traffic_extension_success_collision.png
runs/traffic_extension_route_completion.png
runs/traffic_extension_training_returns.png
runs/traffic_extension_retention.png
```

Generated traffic LaTeX macros are consumed by the public report. Until the long Colab runs exist, the report shows explicit pending text rather than fabricated metrics.

18 Traffic failure taxonomy

Final diagnosis begins with artifact compatibility, not policy interpretation. Every condition must have the declared map, density, respawn mode, fixed test seeds, horizon, episode count, continuous action interface, and reward lineage. Missing or mismatched fingerprints invalidate a comparison even when its headline numbers look plausible.

Once compatibility passes, failures are separated by terminal and behavioral signature:

- High vehicle collision with strong completion indicates useful progress but insufficient interaction handling. Compare density 0.02 and 0.05 stage histories and the reference/safety collision difference.
- High off-road with low vehicle collision suggests geometry forgetting or unstable avoidance. Check traffic-free retention, steering magnitude, and action change before calling it a traffic-only problem.
- High timeout with low collision and low completion indicates a slow or stalled policy. Inspect throttle/brake, speed, return components, and episode length; collision avoidance by non-driving cannot qualify.
- High object collision should be separated from vehicle collision because it may reflect road-boundary or static-scene interaction rather than failure to respond to moving traffic.
- A sharp drop from density 0.05 to 0.10 is stress degradation. It is reported, not used to launch another curriculum stage.

Training curves are read alongside validation outcomes. Critic or return changes without success and completion changes do not establish progress. A late final policy that is worse than an earlier checkpoint tests whether success-first checkpoint retention worked; the saved best model, not the last model, remains the canonical evaluation target. Resource samples distinguish slow simulation from apparent learning stalls but are never used to justify enlarging the network or adding GPU-only work.

Seed disagreement is itself a result. Seed 0 selects a variant under the declared rule; seed 1 tests whether that choice survives a different geometry initialization and SAC random seed. If seed 1 misses Stage A, it remains a failed confirmation. The analysis must not average a failed or unevaluated run as zero, quietly substitute the final model, or promote the other seed-0 variant after seeing test outcomes.

19 Failure handling and interpretation

If Stage A fails, the run is saved with `failed_gate`; Stage B is not started. The success-first Stage-A checkpoint is also promoted to the canonical diagnostic `best_model.zip`, so the exact failure policy can be evaluated after a runtime restart without pretending that the lineage completed adaptation. If one or both seed-0 pilots fail, the protocol records that outcome and does not create another variant. Pilot selection uses only declared metrics. If seed-1 fails, the final result is one completed lineage plus a failed confirmation, not a hidden one-seed average.

Success improvement with collision reduction and limited retention loss supports the transfer hypothesis. Higher success with substantial traffic-free regression instead demonstrates forgetting. A collision reduction achieved by stalling will appear as lower success, timeout, and route completion and will not qualify. Negative findings remain a valid result because the intervention and stopping rule were fixed in advance.

20 Deferred work

Pedestrians, weather, richer maps, pixel observations, constrained RL, safety shields, real-data simulation, broader seeds, and alternative algorithms remain future work. They should not be added to rescue the fixed extension.

Any future study should begin from the final immutable artifacts and state a new research question.

References

[1] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.

[2] Quanyi Li, Zhenghao Peng, Lan Feng, Qihang Zhang, Zhenghai Xue, and Bolei Zhou. Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[3] Anil Ozturk, Mustafa Burak Gunel, Resul Dagdanov, Mirac Ekim Vural, Ferhat Yurdakul, Melih Dal, and Nazim Kemal Ure. Investigating value of curriculum reinforcement learning in autonomous driving under diverse road and weather conditions. *arXiv preprint arXiv:2103.07903*, 2021.

[4] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.